

Capitolo 3

Lambda Calcolo

3.1 Sintassi del Lambda Calcolo

- In matematica una funzione è un caso particolare di relazione. Precisamente, una funzione $f : A \rightarrow B$ è un sottoinsieme di $A \times B$ che a ogni elemento di A associa al più un elemento di B , ossia è un insieme di coppie ordinate, ciascuna formata da un elemento di A e un elemento di B , che gode di una sorta di proprietà di univocità da A verso B . Questa è una visione *estensionale* del concetto di funzione, in quanto si focalizza sull'insieme di coppie ordinate senza considerare il procedimento computazionale tramite il quale si ottiene il risultato $b = f(a) \in B$ partendo dall'argomento $a \in A$.
- Nel 1932 Alonzo Church introdusse il λ -calcolo come parte di una teoria generale delle funzioni e della logica, da usare quale fondamento della matematica. Church era interessato a creare un calcolo che catturasse gli aspetti computazionali delle funzioni, dove per calcolo si intende un sistema formale dotato di una sintassi per generare termini in un certo formato accompagnata da un insieme di regole di riscrittura per trasformare i termini in altri termini. Ciò originò una *visione intensionale* del concetto di funzione, cioè caratterizzata dalle regole computazionali che conducono dall'argomento al risultato.
- Il λ -calcolo descrive le funzioni nella loro piena generalità – incluse funzioni ricorsive e funzioni di ordine superiore (cioè aventi funzioni come argomenti) – attraverso una sintassi estremamente semplice che distingue tra definizione di funzione, detta λ -astrazione, e applicazione di funzione. Questa distinzione consente di superare l'ambiguità che talvolta viene a crearsi in merito alla notazione $f(x)$, usata a volte per denotare la definizione di f e altre volte per denotare il valore che f assume in x .
- Dato un insieme numerabile Var di simboli di variabile, i quali verranno indicati con le lettere x, y, z , l'insieme Λ dei λ -termini è il più piccolo linguaggio su $Var \cup \{\lambda, ., (,)\}$ tale che:
 - $Var \subseteq \Lambda$.
 - $(\lambda x. E) \in \Lambda$ per ogni $x \in Var$ ed $E \in \Lambda$, detta λ -astrazione, dove λ è detto legatore per x in E .
 - $(E_1 E_2) \in \Lambda$ per ogni $E_1, E_2 \in \Lambda$, detta applicazione di E_1 a E_2 .
- Se vediamo la λ -astrazione come un operatore unario prefisso e l'applicazione come un operatore binario infisso, possiamo alternativamente definire Λ come il più piccolo insieme che include Var ed è chiuso rispetto a λ -astrazione e applicazione.
- Per evitare l'uso sistematico delle parentesi assumiamo che l'applicazione abbia precedenza sulla λ -astrazione – così $\lambda x. E_1 E_2$ sta per $(\lambda x. (E_1 E_2))$ – e che la λ -astrazione sia associativa da destra – $\lambda x. \lambda y. E$ sta per $(\lambda x. (\lambda y. E))$ – mentre l'applicazione da sinistra – $E_1 E_2 E_3$ sta per $((E_1 E_2) E_3)$ – da cui la sintassi semplificata $E ::= x \mid \lambda x. E \mid E E$.
- Esempi:
 - $\lambda x. x$ è la funzione identità mentre $\lambda x. \lambda y. x$ e $\lambda x. \lambda y. y$ selezionano uno dei loro due argomenti.
 - $\lambda x. x + 1$ è la funzione successore mentre $\lambda x. x - 1$ è la funzione predecessore.
 - $(\lambda x. x + 1) 2$ è l'applicazione della funzione successore al valore 2.

- Nei precedenti esempi abbiamo usato i simboli $+$, $-$, 1 , 2 sebbene essi non facciano parte della sintassi del λ -calcolo. Ci siamo presi questa libertà perché i numeri naturali e le loro operazioni sono rappresentabili nel λ -calcolo. Ispirandosi a Peano ogni $n \in \mathbb{N}$ è espresso da una funzione $\underline{n}$ detta **numerales di Church** che, dati una funzione s e un argomento z , applica n volte s partendo da z . Precisamente:

- $\underline{0} = \lambda s. \lambda z. z$ così la rappresentazione di 0 restituisce z .
- $\underline{1} = \lambda s. \lambda z. s z$ così la rappresentazione di 1 restituisce una singola applicazione di s a z .
- $\underline{2} = \lambda s. \lambda z. s (s z)$ così la rappresentazione di 2 restituisce una doppia applicazione di s a z .
- $\underline{n} = \lambda s. \lambda z. s (\dots (s z) \dots)$ dove l'applicazione di s è ripetuta n volte come se $\underline{n}$ fosse un iteratore.

- Diciamo che $E' \in \Lambda$ è un sottoterminale di $E \in \Lambda$ sse E' compare in E . Più precisamente, l'insieme $st(E)$ dei sottotermini di $E \in \Lambda$ è definito per induzione sulla struttura sintattica di E come segue:

$$st(E) = \{E\} \cup \begin{cases} \emptyset & \text{se } E \in Var \\ st(E') & \text{se } E \text{ è della forma } \lambda x. E' \\ st(E'_1) \cup st(E'_2) & \text{se } E \text{ è della forma } E'_1 E'_2 \end{cases}$$

- Siano $E \in \Lambda$ e $x \in Var$:

- x occorre in E sse x compare in un sottoterminale di E privo di legatori (cioè privo di λ).
- Un'occorrenza di x in E è legata (a un legatore) se compare in un sottoterminale di E della forma $\lambda x. E'$ – nel qual caso E' è il campo d'azione di quel legatore – altrimenti è libera.
- E è chiuso se ogni occorrenza di ogni variabile che compare in E è legata, altrimenti è aperto.

- Esempi:

- $st(\lambda x. \lambda y. x y) = \{\lambda x. \lambda y. x y, \lambda y. x y, x y, x, y\}$.
- La variabile x non occorre nel termine aperto $\lambda x. y$ perché il solo sottoterminale senza legatori è y .
- Il termine chiuso $(\lambda x. x) \lambda x. x$ ha due occorrenze della stessa variabile che ricadono nel campo d'azione di due legatori che compaiono in due sottotermini indipendenti, mentre in $\lambda x. x (\lambda x. x)$ il campo d'azione del legatore che sta a destra è contenuto in quello del legatore che sta a sinistra.

- Sia $E \in \Lambda$:

- L'insieme $var(E)$ delle variabili che occorrono in E è definito per induzione sulla struttura sintattica di E come segue:

$$var(E) = \begin{cases} \{E\} & \text{se } E \in Var \\ var(E') & \text{se } E \text{ è della forma } \lambda x. E' \\ var(E'_1) \cup var(E'_2) & \text{se } E \text{ è della forma } E'_1 E'_2 \end{cases}$$

- L'insieme $varleg(E)$ delle variabili che occorrono legate in E è definito per induzione sulla struttura sintattica di E come segue:

$$varleg(E) = \begin{cases} \emptyset & \text{se } E \in Var \\ varleg(E') & \text{se } E \text{ è della forma } \lambda x. E' \text{ e } x \notin var(E') \\ varleg(E') \cup \{x\} & \text{se } E \text{ è della forma } \lambda x. E' \text{ e } x \in var(E') \\ varleg(E'_1) \cup varleg(E'_2) & \text{se } E \text{ è della forma } E'_1 E'_2 \end{cases}$$

- L'insieme $varlib(E)$ delle variabili che occorrono libere in E è definito per induzione sulla struttura sintattica di E come segue:

$$varlib(E) = \begin{cases} \{E\} & \text{se } E \in Var \\ varlib(E') \setminus \{x\} & \text{se } E \text{ è della forma } \lambda x. E' \\ varlib(E'_1) \cup varlib(E'_2) & \text{se } E \text{ è della forma } E'_1 E'_2 \end{cases}$$

- Osserviamo che $var(E) = varleg(E) \cup varlib(E)$, ma in generale $varleg(E) \cap varlib(E) \neq \emptyset$ come si vede in $(\lambda x. x) x$. Inoltre E è chiuso quando $varlib(E) = \emptyset$, aperto quando $varlib(E) \neq \emptyset$.

- Siano $E, F \in \Lambda$ e $y \in Var$. Il termine ottenuto da E sostituendo con F ogni occorrenza di y libera in E è definito per induzione sulla struttura sintattica di E come segue:

$$E[F/y] = \begin{cases} F & \text{se } E \in Var \text{ ed } E = y \\ E & \text{se } E \in Var \text{ ed } E \neq y \\ E & \text{se } E \text{ è della forma } \lambda y. E' \\ \lambda x. (E'[F/y]) & \text{se } E \text{ è della forma } \lambda x. E' \text{ e } x \neq y \text{ e } x \notin varlib(F) \\ \lambda z. (E'[z/x][F/y]) & \text{se } E \text{ è della forma } \lambda x. E' \text{ e } x \neq y \text{ e } x \in varlib(F) \text{ con } z \notin var(F) \cup \{y\} \cup var(E') \\ E'_1[F/y] E'_2[F/y] & \text{se } E \text{ è della forma } E'_1 E'_2 \end{cases}$$

dove nella penultima clausola è necessaria una sostituzione preliminare della forma $[z/x]$, con z diversa da tutte le variabili presenti, altrimenti le occorrenze di x libere in F verrebbero erroneamente legate. Ad esempio, il termine aperto $\lambda x. x y$ verrebbe trasformato nel termine chiuso $\lambda x. x x$ se gli fosse direttamente applicata la sostituzione $[x/y]$, mentre la clausola in questione produce $\lambda z. ((x y)[z/x][x/y]) = \lambda z. ((z y)[x/y]) = \lambda z. z x$ che è ancora un termine aperto. ■ftplf_3

3.2 Semantica del Lambda Calcolo e Logica Combinatoria

- La semantica del λ -calcolo è costituita dalle seguenti tre regole di riscrittura di termini che fanno largo uso dell'operazione di sostituzione sintattica di variabili libere:
 - α -conversione: $\lambda x. E =_{\alpha} \lambda y. (E[y/x])$ purché $y \notin \text{varlib}(E)$.
 - β -conversione: $(\lambda x. E) F =_{\beta} E[F/x]$.
 - η -conversione: $\lambda x. E x =_{\eta} E$ purché $x \notin \text{varlib}(E)$.
 - Indichiamo con $\equiv_{\alpha}, \equiv_{\beta}, \equiv_{\eta}$ le corrispondenti relazioni d'equivalenza su Λ indotte dalle chiusure riflessive, simmetriche e transitive delle tre regole rispettivamente. Esse sono congruenze rispetto a λ -astrazione e applicazione, cioè per ogni $\equiv \in \{\equiv_{\alpha}, \equiv_{\beta}, \equiv_{\eta}\}$ da $E_1 \equiv E_2$ segue che:
 - $\lambda x. E_1 \equiv \lambda x. E_2$ per ogni $x \in \text{Var}$.
 - $E_1 F \equiv E_2 F$ per ogni $F \in \Lambda$.
 - $F E_1 \equiv F E_2$ per ogni $F \in \Lambda$.
 - Mentre l'operazione di sostituzione sintattica agisce sulle variabili libere, la regola di α -conversione permette di cambiare il nome delle occorrenze legate di una variabile assieme al corrispondente legatore. Essa è utile per identificare funzioni che differiscono soltanto per il nome delle variabili legate – ad esempio $\lambda x. x =_{\alpha} \lambda y. y$ – e consente di adottare la convenzione secondo cui all'interno di ogni termine i nomi delle variabili legate a occorrenze diverse di legatori siano sempre diversi tra di loro e dai nomi delle variabili libere.
 - La regola di β -conversione, dove $(\lambda x. E) F$ è detto radicale ed $E[F/x]$ è detto ridotto, è la regola fondamentale del λ -calcolo in quanto formalizza l'effetto dell'applicazione di una funzione $\lambda x. E$ a un argomento F sostituendo il parametro effettivo F a ogni occorrenza del parametro formale x nel corpo E della funzione. Nel λ -calcolo vale quanto segue:
 - Ogni funzione con più argomenti è descritta come una funzione di un solo argomento che restituisce una funzione in cui sono gestiti i rimanenti argomenti, un fenomeno che prende il nome di **currying**. Di conseguenza non è necessario passare subito a una funzione tutti gli argomenti di cui necessita. Ad esempio la funzione somma è descritta da $\lambda y. \lambda x. x + y$ e quando viene applicata all'argomento 1 restituisce la funzione successore perché $(\lambda y. \lambda x. x + y) 1 =_{\beta} (\lambda x. x + y)[1/y] = \lambda x. x + 1$.
 - D'altra parte una funzione può essere passata come argomento a un'altra funzione, ossia è possibile trattare **funzioni di ordine superiore**. Ad esempio $(\lambda y. y 2) (\lambda x. x + 1) =_{\beta} (y 2)[\lambda x. x + 1/y] = (\lambda x. x + 1) 2 =_{\beta} (x + 1)[2/x] = 2 + 1 = 3$.
 - La regola di η -conversione, aggiunta successivamente al λ -calcolo, codifica l'uguaglianza estensionale tra due funzioni, cioè il fatto che due funzioni sono uguali sse producono lo stesso risultato quando vengono applicate al medesimo argomento. Essa equivale ad avere una regola che stabilisce che, se per ogni $F \in \Lambda$ risulta $E_1 F \equiv_{\beta} E_2 F$, allora $E_1 = E_2$. Infatti:
 - Da $E_1 F \equiv_{\beta} E_2 F$ per ogni $F \in \Lambda$ segue in particolare $E_1 z \equiv_{\beta} E_2 z$ e quindi $\lambda z. E_1 z \equiv_{\beta} \lambda z. E_2 z$ per $z \notin \text{varlib}(E_1) \cup \text{varlib}(E_2)$, da cui $E_1 = E_2$ applicando la η -conversione ad ambo i membri.
 - Poiché $(\lambda x. E x) F \equiv_{\beta} (E x)[F/x] = E[F/x][F/x] = E F$ per ogni $F \in \Lambda$ purché $x \notin \text{varlib}(E)$, dalla regola di uguaglianza estensionale segue che $\lambda x. E x = E$ purché $x \notin \text{varlib}(E)$.
 - Le regole di β -conversione ed η -conversione prendono rispettivamente il nome di β -riduzione ed η -riduzione quando vengono viste come regole di riscrittura di termini orientate da sinistra a destra:
 - β -riduzione: $(\lambda x. E) F \longrightarrow_{\beta} E[F/x]$.
 - η -riduzione: $\lambda x. E x \longrightarrow_{\eta} E$ purché $x \notin \text{varlib}(E)$.
- Indichiamo con $\longrightarrow_{\beta}^*$ e $\longrightarrow_{\eta}^*$ le rispettive chiusure riflessive e transitive a meno di α -conversione.

- Esempi:

- Posto $\underline{succ} = \lambda n . \lambda x . \lambda y . x (n x y)$, vale $\underline{succ} \underline{n} \longrightarrow_{\beta^*} \underline{n} + 1$. Ad esempio $\underline{succ} \underline{0} = (\lambda n . \lambda x . \lambda y . x (n x y)) (\lambda s . \lambda z . z) \longrightarrow_{\beta} (\lambda x . \lambda y . x (n x y)) [^{\lambda s . \lambda z . z} / n] = \lambda x . \lambda y . x ((\lambda s . \lambda z . z) x y) \longrightarrow_{\beta} \lambda x . \lambda y . x ((\lambda z . z) [^x / s] y) = \lambda x . \lambda y . x ((\lambda z . z) y) \longrightarrow_{\beta} \lambda x . \lambda y . x (z [y / z]) = \lambda x . \lambda y . x y =_{\alpha} \underline{1}$.
- Posto $\underline{add} = \lambda m . \lambda n . \lambda x . \lambda y . m x (n x y)$, vale $\underline{add} \underline{m} \underline{n} \longrightarrow_{\beta^*} \underline{m} + \underline{n}$.
- Posto $\underline{mult} = \lambda m . \lambda n . \lambda x . m (n x)$, vale $\underline{mult} \underline{m} \underline{n} \longrightarrow_{\beta^*} \underline{m} \cdot \underline{n}$.
- Posto $\underline{esp} = \lambda m . \lambda n . m n$, vale $\underline{esp} \underline{m} \underline{n} \longrightarrow_{\beta^*} \underline{n}^m$.
- Posto $\underline{I} = \lambda x . x$, vale $\underline{I} \underline{I} =_{\alpha} (\lambda x . x) (\lambda x' . x') \longrightarrow_{\beta} x [^{\lambda x' . x'} / x] = \lambda x' . x' =_{\alpha} \underline{I}$, cioè $\underline{I} \underline{I} \longrightarrow_{\beta^*} \underline{I}$.
- Posto $\underline{K} = \lambda x . \lambda y . x$, vale $\underline{K} E = (\lambda x . \lambda y . x) E \longrightarrow_{\beta} (\lambda y . x) [^E / x] = \lambda y . E$ e quindi risulta $\underline{K} E F \longrightarrow_{\beta^*} E$ perché $y \notin \text{varlib}(E)$ altrimenti avremmo ridenominato λy nella sostituzione.
- Posto $\underline{S} = \lambda x . \lambda y . \lambda z . x z (y z)$, vale $\underline{S} \underline{K} \underline{K} =_{\alpha} (\lambda x . \lambda y . \lambda z . x z (y z)) (\lambda x' . \lambda y' . x') (\lambda x'' . \lambda y'' . x'') \longrightarrow_{\beta} (\lambda y . \lambda z . x z (y z)) [^{\lambda x' . \lambda y' . x'} / x] (\lambda x'' . \lambda y'' . x'') = (\lambda y . \lambda z . (\lambda x' . \lambda y' . x') z (y z)) (\lambda x'' . \lambda y'' . x'') \longrightarrow_{\beta} (\lambda y . \lambda z . (\lambda y' . x') [^z / x'] (y z)) (\lambda x'' . \lambda y'' . x'') = (\lambda y . \lambda z . (\lambda y' . z) (y z)) (\lambda x'' . \lambda y'' . x'') \longrightarrow_{\beta} (\lambda y . \lambda z . z [^y / y']) (\lambda x'' . \lambda y'' . x'') = (\lambda y . \lambda z . z) (\lambda x'' . \lambda y'' . x'') \longrightarrow_{\beta} (\lambda z . z) [^{\lambda x'' . \lambda y'' . x''} / y] = \lambda z . z =_{\alpha} \underline{I}$, cioè $\underline{S} \underline{K} \underline{K} \longrightarrow_{\beta^*} \underline{I}$.

- I due simboli K ed S sono alla base della logica combinatoria, che come il λ -calcolo aveva l'obiettivo di investigare i fondamenti della matematica usando il concetto base di operazione anziché di insieme. La logica combinatoria venne introdotta nel 1924 da Moses Schönfinkel e fu poi indipendentemente riformulata nel 1930 da Haskell Curry. Diversamente dal λ -calcolo, la logica combinatoria non ha definizioni di funzioni né legatori per le variabili, ma solo i combinatori K ed S , che possono essere visti come funzioni di ordine superiore, e le loro applicazioni. Tuttavia K ed S sono sufficienti per implementare la λ -astrazione e la β -riduzione (variabili e applicazioni sono invece comuni a entrambi i formalismi).

- L'insieme $\mathcal{L}$ dei termini della logica combinatoria è il più piccolo linguaggio su $\text{Var} \cup \{K, S, (,)\}$ tale che:
 - $\text{Var} \subseteq \mathcal{L}$.
 - $K, S \in \mathcal{L}$, detti combinatori.
 - $(PQ) \in \mathcal{L}$ per ogni $P, Q \in \mathcal{L}$, detta applicazione di P a Q .

Assumendo l'applicazione associativa da sinistra, la sintassi semplificata è $P ::= x \mid K \mid S \mid P P$.

- La semantica della logica combinatoria è costituita dalle seguenti regole di riduzione:

- $K P Q \longrightarrow_{\mathcal{L}} P$.
- $S P Q R \longrightarrow_{\mathcal{L}} P R (Q R)$.
- Se $P \longrightarrow_{\mathcal{L}} P'$ allora $P Q \longrightarrow_{\mathcal{L}} P' Q$.
- Se $Q \longrightarrow_{\mathcal{L}} Q'$ allora $P Q \longrightarrow_{\mathcal{L}} P Q'$.

- Dati $P, Q \in \mathcal{L}$ e $y \in \text{Var}$, denotiamo con $\text{var}(P)$ l'insieme delle variabili che occorrono in P – esse sono tutte libere per via dell'assenza di legatori – e con $P[Q/y]$ il termine ottenuto da P sostituendo Q a ogni occorrenza di y in P .

- Teorema: Definiamo i seguenti termini di logica combinatoria che simulano una λ -astrazione in base a tre casi che dipendono dalla forma del corpo della funzione:

- $\hat{\lambda}x . x = S K K$ per ogni $x \in \text{Var}$.
- $\hat{\lambda}x . P = K P$ per ogni $x \in \text{Var}$ e $P \in \mathcal{L}$ tali che $x \notin \text{var}(P)$.
- $\hat{\lambda}x . P_1 P_2 = S (\hat{\lambda}x . P_1) (\hat{\lambda}x . P_2)$ per ogni $x \in \text{Var}$ e $P_1, P_2 \in \mathcal{L}$ tali che $x \notin \text{var}(P_1) \cup \text{var}(P_2)$.

Allora $(\hat{\lambda}x . P) Q \longrightarrow_{\mathcal{L}} P[Q/x]$ per ogni $x \in \text{Var}$ e $P, Q \in \mathcal{L}$.

- Dimostrazione: Procediamo per induzione sulla struttura sintattica di P :

- Se $P = x$ allora $(\hat{\lambda}x . P) Q = S K K Q \longrightarrow_{\mathcal{L}} K Q (K Q) \longrightarrow_{\mathcal{L}} Q = P[Q/x]$.
- Se $x \notin \text{var}(P)$ allora $(\hat{\lambda}x . P) Q = K P Q \longrightarrow_{\mathcal{L}} P = P[Q/x]$.
- Sia $P = P_1 P_2$ con $x \notin \text{var}(P)$ e supponiamo $(\hat{\lambda}x . P_1) Q \longrightarrow_{\mathcal{L}} P_1[Q/x]$ e $(\hat{\lambda}x . P_2) Q \longrightarrow_{\mathcal{L}} P_2[Q/x]$. Allora $(\hat{\lambda}x . P) Q = S (\hat{\lambda}x . P_1) (\hat{\lambda}x . P_2) Q \longrightarrow_{\mathcal{L}} (\hat{\lambda}x . P_1) Q ((\hat{\lambda}x . P_2) Q) \longrightarrow_{\mathcal{L}} P_1[Q/x] P_2[Q/x] = P[Q/x]$. ■ftplf_4

3.3 Ricorsione via Punti Fissi e Calcolabilità

- Le funzioni sono anonime nel λ -calcolo. Pertanto una funzione matematica ricorsiva $f = \dots f \dots$ può essere espressa in λ -calcolo solo in modo non ricorsivo tramite una funzione di ordine superiore $\lambda f. \dots f \dots$ a cui è poi applicato un cosiddetto combinatore di punto fisso Ξ ottenendo $\Xi \lambda f. \dots f \dots$.
- In generale un punto fisso per una funzione $f : A \rightarrow A$ è un elemento $a \in A$ tale che $a = f(a)$, cioè è un elemento di A che è invariante rispetto all'applicazione di f , ovvero è una soluzione dell'equazione $x = f(x)$ e quindi è esprimibile in termini di se stesso. Una funzione può non avere punti fissi, come la funzione successore, oppure può ammetterne soltanto uno o più di uno. Ad esempio, la relazione identità $\mathcal{I}_A = \{(a, a) \mid a \in A\}$ è una funzione per la quale tutti gli elementi di A sono punti fissi.
- Nel λ -calcolo l'equazione di punto fisso per un λ -termine E è espressa come $F \equiv_{\beta\eta} E F$ e si dice che $\Xi \in \Lambda$ è un combinatore di punto fisso sse $\Xi E \equiv_{\beta\eta} E (\Xi E)$ per ogni $E \in \Lambda$. Un combinatore di punto fisso è dunque un λ -termine che consente di calcolare il punto fisso di ogni λ -termine E semplicemente applicando il primo al secondo. Esistono combinatori di punto fisso in λ -calcolo?
- **Teorema (del punto fisso):** Per ogni $E \in \Lambda$ esiste $F \in \Lambda$ tale che $F \equiv_{\beta\eta} E F$.
- Combinatori di punto fisso che dimostrano il teorema:
 - **Combinatore di punto fisso di Turing:** $\Theta = (\lambda x. \lambda y. y (x x y)) (\lambda x. \lambda y. y (x x y))$. Per ogni $E \in \Lambda$ risulta $\Theta E \rightarrow_{\beta} (\lambda y. y ((\lambda x. \lambda y. y (x x y)) (\lambda x. \lambda y. y (x x y)) y)) E \rightarrow_{\beta} E ((\lambda x. \lambda y. y (x x y)) (\lambda x. \lambda y. y (x x y)) E) = E (\Theta E)$.
 - **Combinatore di punto fisso di Curry:** $Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$. Per ogni $E \in \Lambda$ risulta $Y E \rightarrow_{\beta} (\lambda x. E (x x)) (\lambda x. E (x x)) \rightarrow_{\beta} E ((\lambda x. E (x x)) (\lambda x. E (x x))) \equiv_{\beta} E (Y E)$ perché nella seconda β -riduzione $x \notin \text{varlib}(E)$ altrimenti avremmo ridenominato λx in ognuna delle due sostituzioni $(\lambda x. f (x x)) [E/f]$ della prima β -riduzione. Si noti che $Y E \not\rightarrow_{\beta}^* E (Y E)$ perché nel passo finale $E ((\lambda x. E (x x)) (\lambda x. E (x x))) \equiv_{\beta} E (Y E)$ abbiamo sfruttato l'esito della β -riduzione iniziale $Y E \rightarrow_{\beta} (\lambda x. E (x x)) (\lambda x. E (x x))$, non la definizione di Y .
- Esempi (test_0 e $\text{test}_{\leq}$ realizzano un if-then-else basato su $n = 0$ ed $m \leq n$ rispettivamente):
 - Vediamo la definizione della funzione fattoriale e la sua applicazione a uno specifico valore:
 - * Sia $\text{test}_0 = \lambda n. \lambda p. \lambda q. n (\underline{K} q) p$ con $\underline{K} = \lambda x. \lambda y. x$, così $\text{test}_0 0 E F \rightarrow_{\beta} 0 (\underline{K} F) E \rightarrow_{\beta} E$ mentre $\text{test}_0 n E F \rightarrow_{\beta} n (\underline{K} F) E \rightarrow_{\beta} n (\lambda y. F) E \rightarrow_{\beta}^* F$ se $n > 0$ perché $y \notin \text{varlib}(F)$.
 - * Sia $\underline{F} = \lambda r. \lambda n. \text{test}_0 n 1 (\underline{\text{molt}} n (r (\underline{\text{pred}} n)))$.
 - * Allora $\underline{\text{fatt}} = \Theta \underline{F}$.
 - * $\underline{\text{fatt}} 2 = \Theta \underline{F} 2 \rightarrow_{\beta}^* \underline{F} (\Theta \underline{F}) 2 \rightarrow_{\beta}^* \text{test}_0 2 1 (\underline{\text{molt}} 2 (\Theta \underline{F} (\underline{\text{pred}} 2))) \rightarrow_{\beta}^* \underline{\text{molt}} 2 (\Theta \underline{F} 1) \rightarrow_{\beta}^* \underline{\text{molt}} 2 (\underline{F} (\Theta \underline{F}) 1) \rightarrow_{\beta}^* \underline{\text{molt}} 2 (\text{test}_0 1 1 (\underline{\text{molt}} 1 (\Theta \underline{F} (\underline{\text{pred}} 1)))) \rightarrow_{\beta}^* \underline{\text{molt}} 2 (\underline{\text{molt}} 1 (\Theta \underline{F} 0)) \rightarrow_{\beta}^* \underline{\text{molt}} 2 (\underline{\text{molt}} 1 (\underline{F} (\Theta \underline{F}) 0)) \rightarrow_{\beta}^* \underline{\text{molt}} 2 (\underline{\text{molt}} 1 (\text{test}_0 0 1 (\underline{\text{molt}} 0 (\Theta \underline{F} (\underline{\text{pred}} 0))))) \rightarrow_{\beta}^* \underline{\text{molt}} 2 (\underline{\text{molt}} 1 1) \rightarrow_{\beta}^* \underline{\text{molt}} 2 1 \rightarrow_{\beta}^* 2$.
 - L'idea alla base della codifica del predecessore di $n \in \mathbb{N}_{\geq 1}$ è di generare la sequenza di coppie $(0, 0), (0, 1), (1, 2), \dots, (n-1, n)$ per poi restituire la prima componente dell'ultima coppia:
 - * Sia $\underline{c}_{E_1, E_2} = \langle E_1; E_2 \rangle = \lambda z. z E_1 E_2$ dove $z \notin \text{varlib}(E_1) \cup \text{varlib}(E_2)$.
 - * Sia $\underline{gc} = \lambda c. \langle c \underline{\text{pro}}_{2,2}; \underline{\text{succ}} (c \underline{\text{pro}}_{2,2}) \rangle$ dove $\underline{\text{pro}}_{k,i} = \lambda x_1. \dots. \lambda x_k. x_i$ per $k \geq 1, 1 \leq i \leq k$.
 - * Allora $\underline{\text{pred}} = \lambda n. n \underline{gc} \langle 0; 0 \rangle \underline{\text{pro}}_{2,1}$.
 - Di conseguenza possiamo poi definire sottrazione e divisione come segue:
 - * Sia $\underline{S} = \lambda r. \lambda m. \lambda n. \text{test}_0 n m (r (\underline{\text{pred}} m) (\underline{\text{pred}} n))$.
 - * Allora $\underline{\text{sottr}} = \Theta \underline{S}$.
 - * Sia $\underline{\text{test}}_{\leq} = \Theta \underline{T}_{\leq}$ con $\underline{T}_{\leq} = \lambda r. \lambda m. \lambda n. \lambda p. \lambda q. \text{test}_0 m p (\text{test}_0 n q (r (\underline{\text{pred}} m) (\underline{\text{pred}} n) p q))$.
 - * Sia $\underline{D} = \lambda r. \lambda m. \lambda n. \underline{\text{test}}_{\leq} (\underline{\text{succ}} m) n 0 (\underline{\text{succ}} (r (\underline{\text{sottr}} m n) n))$ dove $m+1 \leq n$ significa $m < n$.
 - * Allora $\underline{\text{div}} = \Theta \underline{D}$.

- Tra gli anni 1880 e gli anni 1930 vennero pubblicati molti studi dedicati alla ricorsione e al suo uso nell'ambito dei numeri naturali, tra cui citiamo quelli di Dedekind, Peano, Skolem, Hilbert, Ackermann, Peter, Herbrand e Gödel. Nel 1936 Stephen Kleene dimostrò che le funzioni ricorsive generali di Gödel ed Herbrand sono definibili mediante λ -termini. Nel 1937 Alan Turing dimostrò che le funzioni definibili mediante λ -termini sono calcolabili da macchine di Turing. I due risultati insieme stabiliscono che una funzione $f : \mathbb{N} \rightarrow \mathbb{N}$ è calcolabile da una macchina di Turing sse f è definibile mediante un λ -termine, ovvero sse f è una funzione ricorsiva generale. Queste sono tutte le funzioni calcolabili?
- Chiamiamo metodo effettivo una sequenza finita di istruzioni interpretabili senza ambiguità la cui esecuzione termina sempre in un tempo finito producendo il risultato corretto. Non tutte le funzioni su $\mathbb{N}$ sono calcolabili attraverso un metodo effettivo; poiché $\mathbb{Z}$ e $\mathbb{Q}$ sono equipotenti a $\mathbb{N}$, e quindi tutti gli interi e tutti i razionali sono codificabili come naturali, ciò vale anche per le funzioni su $\mathbb{Z}$ e su $\mathbb{Q}$. Ad esempio, dato un qualsiasi polinomio $p(x_1, x_2, \dots, x_n)$ a coefficienti interi di grado arbitrario, il decimo problema di Hilbert richiede di stabilire se l'equazione $p(x_1, x_2, \dots, x_n) = 0$ ammette soluzioni intere. Poiché tale problema è stato dimostrato essere indecidibile, la funzione su $\mathbb{Z}$ che restituisce 1 o 0 a seconda che l'equazione $p(x_1, x_2, \dots, x_n) = 0$ ammetta soluzioni intere o meno non è calcolabile.
- I risultati precedentemente citati di Kleene e Turing hanno indotto a ritenere che l'insieme delle funzioni calcolabili tramite un qualsiasi metodo effettivo sia equivalentemente caratterizzato dai concetti di Turing-calcolabilità, λ -definibilità e ricorsione generale, come espresso dalle seguenti tesi:
 - **Tesi di Church**: Le funzioni su $\mathbb{N}$ che sono calcolabili attraverso un qualsiasi metodo effettivo sono esattamente quelle definibili mediante λ -termini.
 - **Tesi di Turing**: Le funzioni su $\mathbb{N}$ che sono calcolabili attraverso un qualsiasi metodo effettivo sono esattamente quelle calcolabili mediante macchine di Turing.
- Approfondiamo ora il legame tra funzioni ricorsive generali e funzioni definibili mediante λ -termini, evidenziando il ruolo dei combinatori di punto fisso per catturare la ricorsione generale.
- L'insieme delle funzioni ricorsive primitive è composto dalle seguenti funzioni base su $\mathbb{N}$:
 - $0_k(x_1, \dots, x_k) = 0$, detta funzione zero su $k \geq 0$ argomenti;
 - $\text{succ}(x) = x + 1$, detta funzione successore;
 - $\text{pro}_{k,i}(x_1, \dots, x_k) = x_i$, detta funzione proiezione i -esima su $k \geq 1$ argomenti, dove $1 \leq i \leq k$;
 ed è chiuso rispetto alle seguenti operazioni:
 - composizione: se $h(y_1, \dots, y_m)$ è ricorsiva primitiva e $g_i(x_1, \dots, x_k)$ è ricorsiva primitiva per ogni $1 \leq i \leq m$, allora anche $f(x_1, \dots, x_k) = h(g_1(x_1, \dots, x_k), \dots, g_m(x_1, \dots, x_k))$ è ricorsiva primitiva;
 - ricorsione primitiva: se $h(x_1, \dots, x_m)$ e $g(y, k, x_1, \dots, x_m)$ sono ricorsive primitive, allora anche $f(k, x_1, \dots, x_m)$ definita ponendo:

$$f(0, x_1, \dots, x_m) = h(x_1, \dots, x_m)$$

$$f(k+1, x_1, \dots, x_m) = g(f(k, x_1, \dots, x_m), k, x_1, \dots, x_m)$$
 è ricorsiva primitiva (h rappresenta il caso base, g quello induttivo).
- Molte funzioni di largo uso sono ricorsive primitive:
 - $1_k(x_1, \dots, x_k) = \text{succ}(0_k(x_1, \dots, x_k))$,
 $2_k(x_1, \dots, x_k) = \text{succ}(1_k(x_1, \dots, x_k))$, e così via.
 - $\text{add}(0, x) = \text{pro}_{1,1}(x)$,
 $\text{add}(k+1, x) = g(\text{add}(k, x), k, x)$ dove $g(y, k, x) = \text{succ}(\text{pro}_{3,1}(y, k, x))$.
 - $\text{molt}(0, x) = 0_1(x)$,
 $\text{molt}(k+1, x) = g(\text{molt}(k, x), k, x)$ dove $g(y, k, x) = \text{add}(\text{pro}_{3,1}(y, k, x), \text{pro}_{3,3}(y, k, x))$.
 - $\text{esp}(0, x) = 1_1(x)$,
 $\text{esp}(k+1, x) = g(\text{esp}(k, x), k, x)$ dove $g(y, k, x) = \text{molt}(\text{pro}_{3,1}(y, k, x), \text{pro}_{3,3}(y, k, x))$.
 - $\text{fatt}(0) = 1_0$,
 $\text{fatt}(k+1) = g(\text{fatt}(k), k)$ dove $g(y, k) = \text{molt}(\text{pro}_{2,1}(y, k), g'(y, k))$ con $g'(y, k) = \text{succ}(\text{pro}_{2,2}(y, k))$.

- Una funzione $f(k, x_1, \dots, x_m)$ ottenuta per ricorsione primitiva da $h(x_1, \dots, x_m)$ e $g(y, k, x_1, \dots, x_m)$ non è intrinsecamente ricorsiva perché può essere implementata in modo iterativo. In particolare, il valore che f assume in $k \in \mathbb{N}$ dati i valori di $x_1, \dots, x_m$ coincide col valore f' calcolato come segue:
 - Inizializzare f' ponendo $f' = h(x_1, \dots, x_m)$.
 - Inizializzare f'' ponendo $f'' = g(f', 0, x_1, \dots, x_m)$.
 - Per ogni i da 1 a k ripetere:
 - * Aggiornare f' ponendo $f' = f''$.
 - * Aggiornare f'' ponendo $f'' = g(f', i, x_1, \dots, x_m)$.

- È quindi ragionevole aspettarsi che non tutte le funzioni ricorsive siano esprimibili come funzioni ricorsive primitive. Un controesempio è dato dalla funzione di Ackermann:

$$\begin{aligned} A(0, 0, y) &= y \\ A(0, x+1, y) &= A(0, x, y) + 1 \\ A(1, 0, y) &= 0 \\ A(k+2, 0, y) &= 1 \\ A(k+1, x+1, y) &= A(k, A(k+1, x, y), y) \end{aligned}$$

detta esponenziale generalizzata perché cresce più rapidamente di qualsiasi funzione ricorsiva primitiva:

$$\begin{aligned} A(0, x, y) &= x + y \\ A(1, x, y) &= x \cdot y \\ A(2, x, y) &= y^x \\ A(3, x, y) &= y^{y^{\dots^y}} \text{ dove il numero di esponenti } y \text{ è pari a } x \end{aligned}$$

- Una funzione $f(x_1, \dots, x_m)$ è ricorsiva generale sse viene ottenuta per minimizzazione da una funzione ricorsiva primitiva $h(k, x_1, \dots, x_m)$, cioè $f(x_1, \dots, x_m) = \min\{k \in \mathbb{N} \mid h(k, x_1, \dots, x_m) = 0\}$.
- Questi sono i λ -termini che codificano le funzioni ricorsive generali, nel senso che se $f(n_1, \dots, n_k) = m$ allora $\underline{f} \underline{n}_1 \dots \underline{n}_k \longrightarrow_{\beta^*} \underline{m}$ per ogni funzione ricorsiva generale f e per ogni $k, n_1, \dots, n_k, m \in \mathbb{N}$:

– Funzioni base:

- * $\underline{0}_k = \lambda x_1. \dots \lambda x_k. \lambda s. \lambda z. z$.
- * $\underline{succ} = \lambda n. \lambda x. \lambda y. x (n x y)$.
- * $\underline{pro}_{k,i} = \lambda x_1. \dots \lambda x_k. x_i$.

– Composizione:

- * $\underline{f} = \lambda x_1. \dots \lambda x_k. \underline{h} (\underline{g}_1 x_1 \dots x_k) \dots (\underline{g}_m x_1 \dots x_k)$.

– Ricorsione primitiva:

- * L'idea della codifica è di generare come nella versione iterativa la sequenza di $k+1$ triple:

$$\begin{aligned} (0; h(x_1, \dots, x_m); g(h(x_1, \dots, x_m), 0, x_1, \dots, x_m)) &= (0; f(0, x_1, \dots, x_m); f(1, x_1, \dots, x_m)) \\ (1; f(1, x_1, \dots, x_m); g(f(1, x_1, \dots, x_m), 1, x_1, \dots, x_m)) &= (1; f(1, x_1, \dots, x_m); f(2, x_1, \dots, x_m)) \\ &\dots = \dots \\ (k; f(k, x_1, \dots, x_m); g(f(k, x_1, \dots, x_m), k, x_1, \dots, x_m)) &= (k; f(k, x_1, \dots, x_m); f(k+1, x_1, \dots, x_m)) \end{aligned}$$
 per poi restituire la seconda componente dell'ultima tripla generata.
- * Sia $\underline{t}_{E_1, E_2, E_3} = \langle E_1; E_2; E_3 \rangle = \lambda z. z E_1 E_2 E_3$ dove $z \notin \text{varlib}(E_1) \cup \text{varlib}(E_2) \cup \text{varlib}(E_3)$.
- * Notiamo che $\underline{t}_{E_1, E_2, E_3} \underline{pro}_{3,1} \longrightarrow_{\beta^*} E_1$, $\underline{t}_{E_1, E_2, E_3} \underline{pro}_{3,2} \longrightarrow_{\beta^*} E_2$, $\underline{t}_{E_1, E_2, E_3} \underline{pro}_{3,3} \longrightarrow_{\beta^*} E_3$.
- * Sia $\underline{gt} = \lambda t. \langle \underline{succ} (\underline{t} \underline{pro}_{3,1}); \underline{t} \underline{pro}_{3,3}; \underline{g} (\underline{t} \underline{pro}_{3,3}) (\underline{succ} (\underline{t} \underline{pro}_{3,1})) x_1 \dots x_m \rangle$.
- * Allora $\underline{f} = \lambda k. \lambda x_1. \dots \lambda x_m. k \underline{gt} \langle \underline{0}; \underline{h} x_1 \dots x_m; \underline{g} (\underline{h} x_1 \dots x_m) \underline{0} x_1 \dots x_m \rangle \underline{pro}_{3,2}$.

– Ricorsione generale:

- * L'idea della codifica è di generare la sequenza $h(0, x_1, \dots, x_m)$, $h(1, x_1, \dots, x_m)$, e così via fino a incontrare il primo valore pari a 0 nella sequenza se esiste. La funzione che genera tale sequenza è il punto fisso di un'opportuna funzione di ordine superiore.
- * Sia $\underline{H} = \lambda r. \lambda k. \lambda x_1. \dots \lambda x_m. \underline{test}_0 (\underline{h} k x_1 \dots x_m) k (r (\underline{succ} k) x_1 \dots x_m)$.
- * Allora $\underline{f} = \lambda x_1. \dots \lambda x_m. \Theta \underline{H} \underline{0} x_1 \dots x_m$.

3.4 Terminazione e Confluenza nel Lambda Calcolo

- L'assenza di radicali in un λ -termine fornisce una chiara evidenza della finalit  del termine stesso dal punto di vista computazionale, cos  come il numero ottenuto da un'espressione aritmetica dopo aver calcolato tutte le sue operazioni ne rappresenta il valore finale. La procedura di β -riduzione del λ -calcolo non   per  terminante, perch  la sua applicazione non fa necessariamente diminuire il numero di radicali. Basta prendere $\underline{\omega} = \lambda x . x x$ dato che $\underline{\omega} = \underline{\omega} \underline{\omega} = (\lambda x . x x) \underline{\omega} \longrightarrow_{\beta} \underline{\omega} \underline{\omega} \longrightarrow_{\beta} \dots \longrightarrow_{\beta} \underline{\omega} \underline{\omega} \longrightarrow_{\beta} \dots$
- Un λ -termine E   in forma normale sse non contiene radicali, cio  sse la β -riduzione non   applicabile ad esso, ovvero sse   generato dalla sintassi $N ::= x \mid \lambda x . N \mid x F$ dove $F ::= x \mid \lambda x . N \mid F F$. Diciamo poi che $E \in \Lambda$ ammette forma normale sse esiste $E' \in \Lambda$ in forma normale tale che $E \longrightarrow_{\beta}^* E'$. Non tutti i λ -termini ammettono forma normale, come dimostra l'autoapplicazione $\underline{\omega}$.
- La procedura di β -riduzione del λ -calcolo non   deterministica, nel senso che non   definita a priori una strategia di riduzione che, in presenza di pi  radicali in un termine, stabilisca a quale radicale applicare la β -riduzione per primo. In via di principio non   pertanto scontato che si riesca a raggiungere la forma normale per un termine che la ammette, n  che la forma normale sia unica quando esiste.
- In presenza di pi  radicali, si pu  scegliere tra ridurre quelli pi  esterni o quelli pi  interni rispetto alla struttura sintattica del termine. Per esempio, in $(\lambda x . E) F$ si pu  applicare la β -riduzione all'intero termine prima oppure dopo averla applicata dentro E e dentro F .
- Se ci sono pi  radicali allo stesso livello nella struttura sintattica del termine, si pu  scegliere tra ridurre quello pi  a sinistra oppure quello pi  a destra. Per esempio, in $E_1 E_2$ dove E_1 non   una λ -astrazione, si pu  applicare la β -riduzione prima dentro E_1 oppure prima dentro E_2 .
- Strategie di riduzione comunemente utilizzate:
 - Chiamata per nome: viene sistematicamente ridotto il radicale pi  a sinistra tra quelli pi  esterni nella struttura sintattica del termine, che corrisponde a passare gli argomenti alla funzione senza valutarli. Ad esempio, $\underline{\omega} (\underline{I} \underline{I}) = (\lambda x . x x) (\underline{I} \underline{I}) \longrightarrow_{\beta} (\underline{I} \underline{I}) (\underline{I} \underline{I}) \longrightarrow_{\beta} \underline{I} (\underline{I} \underline{I}) \longrightarrow_{\beta} \underline{I} \underline{I} \longrightarrow_{\beta} \underline{I}$.
 - Chiamata per valore: viene sistematicamente ridotto il radicale pi  a sinistra tra quelli pi  interni nella struttura sintattica del termine, che corrisponde a valutare gli argomenti prima di passarli alla funzione. Ad esempio, $\underline{\omega} (\underline{I} \underline{I}) = \underline{\omega} ((\lambda x . x) \underline{I}) \longrightarrow_{\beta} \underline{\omega} \underline{I} \longrightarrow_{\beta} \underline{I} \underline{I} \longrightarrow_{\beta} \underline{I}$.
- Quando la forma normale esiste, la sua unicit  pu  essere garantita dalla propriet  di confluenza, la quale pu  essere formulata in modi diversi:
 - Propriet  di confluenza forte o del diamante: per ogni $E \in \Lambda$, se $E \longrightarrow_{\beta} E_1$ ed $E \longrightarrow_{\beta} E_2$, allora esiste $E' \in \Lambda$ tale che $E_1 \longrightarrow_{\beta} E'$ ed $E_2 \longrightarrow_{\beta} E'$.
 - Propriet  di confluenza debole o di Church-Rosser: per ogni $E \in \Lambda$, se $E \longrightarrow_{\beta}^* E_1$ ed $E \longrightarrow_{\beta}^* E_2$, allora esiste $E' \in \Lambda$ tale che $E_1 \longrightarrow_{\beta}^* E'$ ed $E_2 \longrightarrow_{\beta}^* E'$.

La propriet  di confluenza forte implica quella debole, ma la prima non vale nel λ -calcolo come si vede dall'applicazione delle due strategie di riduzione a $\underline{\omega} (\underline{I} \underline{I})$.

- Lemma (strip lemma): Per ogni $E \in \Lambda$, se $E \longrightarrow_{\beta} E_1$ ed $E \longrightarrow_{\beta}^* E_2$, allora esiste $E' \in \Lambda$ tale che $E_1 \longrightarrow_{\beta}^* E'$ ed $E_2 \longrightarrow_{\beta}^* E'$.
- Teorema (di Church-Rosser): Il λ -calcolo gode della propriet  di confluenza debole.
- Corollario: Se $E \in \Lambda$ ammette forma normale, allora questa   unica a meno di α -conversione.
- Corollario: La teoria della β -equivalenza   coerente, cio  non vale che $E_1 \equiv_{\beta} E_2$ per ogni $E_1, E_2 \in \Lambda$.

- Curry dimostrò che, quando esiste, la forma normale viene sempre raggiunta con la strategia di chiamata per nome. Ciò non vale per la strategia di chiamata per valore. Per esempio, il termine $(\lambda y. z)(\underline{\omega} \underline{\omega})$ β -riduce alla forma normale z con la prima strategia, mentre β -riduce a se stesso con la seconda.
- Di contro, quando conduce alla forma normale, la strategia di chiamata per valore potrebbe risultare più efficiente della strategia di chiamata per nome. Per esempio, il termine $(\lambda n. \underline{add} \ n \ n)(\underline{molt} \ 5 \ 4)$ β -riduce più rapidamente a $\underline{40}$ con la prima perché la seconda calcola due volte $\underline{molt} \ 5 \ 4$.
- Per coniugare la raggiungibilità della forma normale garantita dalla strategia di chiamata per nome con la maggiore efficienza della strategia di chiamata per valore, conviene considerare sistematicamente solo i radicali al primo livello. Ciò conduce alla cosiddetta forma normale di testa debole, la quale è una forma normale di testa oppure una λ -astrazione.
- Il formato di una forma normale di testa è $\lambda x_1. \dots \lambda x_n. y E_1 \dots E_k$ – dove $n, k \in \mathbb{N}$ e la variabile di testa y può essere diversa da $x_1, \dots, x_n$ – in cui le sole β -riduzioni possibili sono interne ai singoli sottotermini E_i data la presenza della variabile y all’inizio e l’associatività da sinistra dell’applicazione. Il termine più semplice in questo formato è y .
- La forma normale di testa può essere vista come un’approssimazione della forma normale. Ciò deriva dal fatto che un termine in forma normale è anche in forma normale di testa (perché se non ha radicali allora in particolare non ne ha all’inizio) e dalla seguente proprietà di risolubilità: $E \in \Lambda$ ha forma normale di testa sse esiste $h \in \mathbb{N}$ tale che $E E_1 \dots E_h \equiv_\beta \underline{I}$, ossia $E E_1 \dots E_h F \equiv_\beta F$ per ogni $F \in \Lambda$.
- Se un λ -termine non è in forma normale di testa allora è nel formato $\lambda x_1. \dots \lambda x_n. (\lambda y. E_0) E_1 \dots E_k$ con $k \geq 1$, dove $(\lambda y. E_0) E_1$ è detto radicale di testa ed è il termine più semplice in questo formato. Quando $n \geq 1$, cioè il radicale di testa è preceduto da almeno un legatore, tale formato è un esempio di λ -astrazione che rappresenta una forma normale di testa debole che non è in forma normale di testa, nel qual caso $(\lambda y. E_0) E_1 \dots E_k$ costituisce il prossimo livello al quale applicare la β -riduzione.
- In alternativa, si può adattare la strategia di chiamata per nome ragionando in termini di riscrittura di grafi anziché riscrittura di termini come scoperto nel 1971 da Christopher Wadsworth. Precisamente, se si considera l’albero di sintassi astratta di un λ -termine, è possibile individuare eventuali sottotermini ripetuti e rimpiazzare i rispettivi sottoalberi con un unico sottoalbero condiviso da tutti gli archi entranti nelle radici di quei sottoalberi – rendendo l’albero originario un grafo – in modo da valutare quei sottotermini una sola volta.
- La strategia risultante viene detta strategia di chiamata per necessità o valutazione pigra, in quanto la valutazione degli argomenti di una funzione viene ritardata il più possibile, e memorizza i valori degli argomenti per usi successivi, così da non doverli ricalcolare. Ad esempio, in $\lambda n. \underline{add} \ n \ n$ si avrebbe un unico sottoalbero condiviso per n e quindi $\underline{molt} \ 5 \ 4$ verrebbe calcolato una sola volta.

3.5 Lambda Calcolo con Tipi

- I tipi vennero introdotti tra il 1910 e il 1913 da Russell e Whitehead nel loro libro “Principia Mathematica” per evitare i paradossi logici. I tipi permettono di classificare i termini, cioè di individuare insiemi di termini con proprietà computazionali simili, così da prevenire comportamenti indesiderati come ad esempio la non terminazione. Essi sono necessari anche in λ -calcolo (e in logica combinatoria) per evitare alcuni paradossi, come quello scoperto da Kleene e Rosser il quale essenzialmente replica il paradosso di Richard in teoria dei linguaggi formali e poi venne riformulato da Church.
- Consideriamo un insieme T formato da un insieme numerabile di simboli di tipo – i quali verranno indicati con le lettere τ, σ, ρ – e chiuso rispetto al tipo funzione $\tau \rightarrow \sigma$, dove τ rappresenta il tipo dell’argomento e σ il tipo del risultato. Per evitare l’uso sistematico delle parentesi assumiamo che l’operatore $\rightarrow$ sui tipi sia associativo da destra come la λ -astrazione per coerenza col currying.

- Nel λ -calcolo esistono due diversi generi di sistemi di tipi:
 - I sistemi di tipi alla Church stabiliscono che ogni λ -termine venga definito assieme al suo tipo cosicché sintassi e semantica includono il controllo dei tipi.
 - I sistemi di tipi alla Curry stabiliscono che il tipo di ogni λ -termine sia attribuito mediante regole formali che inferiscono il tipo dal formato del termine.
- L'insieme Λ' dei λ -termini con tipi alla Church è il più piccolo linguaggio su $Var \cup \{\lambda, ., (,)\} \cup \mathbb{T}$ tale che:
 - $x^\tau \in \Lambda'$ per ogni $x \in Var$ e $\tau \in \mathbb{T}$.
 - $(\lambda x^\tau. E^\sigma)^{\tau \rightarrow \sigma} \in \Lambda'$ per ogni $x^\tau, E^\sigma \in \Lambda'$.
 - $(E_1^{\tau \rightarrow \sigma} E_2^\tau)^\sigma \in \Lambda'$ per ogni $E_1^{\tau \rightarrow \sigma}, E_2^\tau \in \Lambda'$.
- Le regole semantiche su Λ' si modificano di conseguenza includendo il controllo dei tipi come segue:
 - α -conversione con tipi: $\lambda x^\tau. E^\sigma =_\alpha \lambda y^\tau. (E^\sigma[y^\tau/x^\tau])$ purché $y^\tau \notin varlib(E^\sigma)$.
 - β -conversione con tipi: $(\lambda x^\tau. E^\sigma) F^\tau =_\beta E^\sigma[F^\tau/x^\tau]$.
 - η -conversione con tipi: $\lambda x^\tau. E^\sigma x^\tau =_\eta E^\sigma$ purché $x^\tau \notin varlib(E^\sigma)$.
- Nel caso dei sistemi di tipi alla Curry, sintassi e semantica del λ -calcolo non cambiano, ma si aggiunge un sistema di attribuzione dei tipi. Data una base $\Gamma = \{x_i : \tau_i \mid x_i \in Var, \tau_i \in \mathbb{T}, 1 \leq i \leq n, x_i \neq x_j \text{ se } i \neq j\}$ di dichiarazioni di tipo per le variabili, il sistema consiste nelle seguenti tre regole di inferenza dei tipi dove $\Gamma \vdash E : \tau$ significa che, partendo da Γ , al λ -termine E viene attribuito il tipo τ :

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \quad \frac{\Gamma \cup \{x : \tau\} \vdash E : \sigma}{\Gamma \vdash (\lambda x. E) : \tau \rightarrow \sigma} \quad \frac{\Gamma \vdash E_1 : \tau \rightarrow \sigma \quad \Gamma \vdash E_2 : \tau}{\Gamma \vdash (E_1 E_2) : \sigma}$$
- Teorema: $E^\tau \in \Lambda'$ sse $\{x_i : \tau_i \mid x_i^{\tau_i} \in varlib(E^\tau)\} \vdash |E^\tau| : \tau$, dove $|E^\tau|$ è il λ -termine ottenuto da E^τ eliminando tutti i tipi presenti in esso.
- Ogni termine in Λ' è fortemente normalizzabile, cioè qualsiasi strategia di riduzione applicata ad esso termina e produce la sua forma normale. Di conseguenza $\Lambda' \subsetneq \Lambda$, ossia non tutti i λ -termini possono essere estesi coi tipi. Per esempio, la funzione identità è definibile in Λ' come $\lambda x^\tau. x^\tau$, mentre $\underline{\omega} = \lambda x. x x$ non lo è perché x dovrebbe essere contemporaneamente di tipo $\tau \rightarrow \sigma$ e di tipo τ .
- Nemmeno i combinatori di punto fisso Θ e Y sono definibili in Λ' . Tuttavia, poiché sussiste la necessità di implementare la ricorsione, Λ' viene esteso con delle costanti – così da poter trattare anche i numeri senza dover ricorrere ai numerali di Church – tra le quali F_τ di tipo $(\tau \rightarrow \tau) \rightarrow \tau$ per ogni $\tau \in \mathbb{T}$, assieme alla seguente ulteriore regola di punto fisso:
 - δ -conversione: $F_\tau E^{\tau \rightarrow \tau} =_\delta E^{\tau \rightarrow \tau} (F_\tau E^{\tau \rightarrow \tau})$.
- Esempi di funzioni definibili in Λ' :
 - $\underline{I} = \lambda x. x$ è definibile in Λ' con tipo $\tau \rightarrow \tau$ usando x^τ .
 - $\underline{K} = \lambda x. \lambda y. x$ è definibile in Λ' con tipo $\tau \rightarrow \sigma \rightarrow \tau$ usando x^τ e y^σ .
 - $\underline{S} = \lambda x. \lambda y. \lambda z. x z (y z)$ è definibile in Λ' con tipo $(\tau \rightarrow \sigma \rightarrow \rho) \rightarrow (\tau \rightarrow \sigma) \rightarrow \tau \rightarrow \rho$ usando $x^{\tau \rightarrow \sigma \rightarrow \rho}, y^{\tau \rightarrow \sigma}, z^\tau$.
 - $\underline{n} = \lambda s. \lambda z. s (\dots (s z) \dots)$ è definibile in Λ' con tipo $\nu = (\tau \rightarrow \tau) \rightarrow \tau \rightarrow \tau$ usando $s^{\tau \rightarrow \tau}$ e z^τ .
 - $\underline{succ} = \lambda n. \lambda x. \lambda y. x (n x y)$ è definibile in Λ' con tipo $\nu \rightarrow \nu$ usando $n^\nu, x^{\tau \rightarrow \tau}, y^\tau$.
 - $\underline{add} = \lambda m. \lambda n. \lambda x. \lambda y. m x (n x y)$ e $\underline{molt} = \lambda m. \lambda n. \lambda x. m (n x)$ sono entrambe definibili in Λ' con tipo $\nu \rightarrow \nu \rightarrow \nu$ usando $m^\nu, n^\nu, x^{\tau \rightarrow \tau}, y^\tau$.
 - $\underline{esp} = \lambda m. \lambda n. m n$ è definibile in Λ' con tipo $(\tau \rightarrow \sigma) \rightarrow \tau \rightarrow \sigma$ usando $m^{\tau \rightarrow \sigma}$ ed n^τ .

- Le regole di inferenza dei tipi alla Curry sono interessanti perché, oltre a coincidere con le regole di inferenza per il connettivo di implicazione della logica intuizionista di Heyting (in cui non vale il principio del terzo escluso) se si omettono i λ -termini (notare che la terza regola è il modus ponens), esse danno luogo al cosiddetto isomorfismo di Curry-Howard o analogia formule-tipi secondo cui:

- I tipi corrispondono alle formule. Notiamo che tutti i tipi negli esempi di cui sopra sono tautologie, in particolare i tipi di $\underline{K}$ ed $\underline{S}$ sono assiomi usati nei sistemi deduttivi alla Hilbert (v. Sez. 5.5).
- I λ -termini di quei tipi corrispondono alle dimostrazioni della validità di quelle formule.
- La β -riduzione corrisponde alla composizione delle dimostrazioni basata sulla regola del taglio,

perché se da un lato si deriva $\frac{\Gamma \cup \{x : \tau\} \vdash E : \sigma}{\Gamma \vdash (\lambda x . E) : \tau \rightarrow \sigma}$ e dall'altro $\Gamma \vdash F : \tau$ allora $\Gamma \vdash (\lambda x . E) F : \sigma$. ■ftplf_6

